

Developer Guide

This document is the day-to-day “how do I work in this codebase” reference. For the authoritative architecture overview (package layout, config system, ingestion pipelines), see `CLAUDE.md` at the repo root — this guide doesn’t duplicate that; it walks through concrete developer workflows instead.

Package layout

All source lives under `src/`, using MATLAB packages (+-prefixed directories):

```
src/
├── +migration/
│   └── apply_known_fixes.m          # Category C pre-validation corrections
├── +narwc/
│   ├── +db/
│   │   ├── Connection.m           # DB connection wrapper (fetch/execute/insert/update, tra
│   │   └── FieldDefinitions.m     # Canonical source of truth for all 55 field names/types
│   ├── +io/
│   │   ├── DataTypeConverter.m    # Type coercion before upload
│   │   └── +parsers/
│   │       ├── BaseParser.m       # Abstract base class
│   │       └── StandardFormat.m   # Reference parser for the legacy CSV (no header, comma-
delimited)
│   │           ├── NEAQFormat.m    # Reference parser for a per-contributor header-based format
│   │           ├── TemplateFormat.m # Copy this to add a new contributor parser
│   │           └── ParserFactory.m # Explicit createByName() selection –
no auto-detection
│   └── +ingestion/
│       ├── SurveyExtractor.m      # One-
time legacy CSV split (migration)
│       └── SurveyFileWriter.m     # Shared per-
FILEID chunk writer (both pipelines)
│   ├── convert_contributor_batch.m # Routine ingestion: parse + split a contributor'
│   ├── run_batch_upload.m         # Shared connect→upload→stats→close
│   └── BatchUploader.m            # Validates + uploads survey CSVs to SQL Server, transa
safe
├── +validation/
│   ├── SurveyValidator.m         # Orchestrates rule modules + override matching
│   └── FieldValidator.m          # Static field-
level validators
└── ErrorCollector.m             # Accumulates errors/warnings/info with rule_id + event
```

```

|   └─ +rules/                # One function per rule module (see below)
|   └─ +processing/
|   └─ SurveyProcessor.m      # Runs ordered steps from +steps/
|       └─ ChangeTracker.m    # Field-
level change tracking for audit reports
|   └─ +steps/                # Individual processing steps
|   └─ +reports/
|   └─ ValidationReport.m, ProcessingReport.m, SummaryStatistics.m # Markdown report
|   └─ +utils/
|       └─ sanitize_filename.m

```

lib/+logging/ is a third-party logging toolbox—use `logging.Logger('narwc.<package>.<class>')` for consistent log naming.

Typical workflow: validating and uploading a survey

```

% Parse a raw file with an explicit, known parser (no auto-detection)
parser = narwc.io.parsers.ParserFactory.createByName('StandardFormat');
[data, metadata] = parser.parse('data/legacy/original_csv/survey.csv');

% Validate
config = load_config('migration'); % or load_config() for strict defaults
validator = narwc.validation.SurveyValidator(config.validation);
[is_valid, results] = validator.validate(data);

% results.summary has errors/warnings_new/warnings_acknowledged counts;
% results.errors / results.warnings are struct arrays with field, rule_id,
% severity, eventno, message (see ErrorCollector.m).

if is_valid
    conn = narwc.db.Connection.create();
    uploader = narwc.ingestion.BatchUploader(conn, base_dir, 'Config', config);
    uploader.uploadSurvey(data);
    conn.close();
end

```

In practice you rarely call `BatchUploader` directly — both pipelines drive it through `narwc.ingestion.run_batch_upload()`, which handles the connect/upload/stats/close sequence (see `scripts/migration/step2_upload_surveys.m` and `scripts/ingestion/upload_contributor_batch.m` for the two callers).

Adding a new validation rule

1. Copy `src/+narwc/+validation/+rules/_rules_template.m` to a new file in the same directory.

2. Implement it — signature is `fn(data, collector, config)`, where `config` is the sub-struct for that rule's domain (e.g. `config.species`), already merged from defaults before the call.
3. Add any tunable parameters to `config/defaults/validation_config_default.m` — never hardcode a default inside the rule file, since that's invisible to anyone editing config files.
4. Register it in `SurveyValidator.runValidationRules()` with a corresponding `obj.config.validate_<name>` flag, and add that flag's default to `SurveyValidator.defaultConfig()`.
5. Give every error/warning a stable `rule_id` (e.g. `my_rules.check_name`) via `collector.addError(field, row, message, severity, rule_id, eventno)` — `rule_id` is what curators reference in override CSVs and what the migration-statistics tooling (`scripts/migration/step3_validate_migration.m`) aggregates by.

See `docs/validation_rules_guide.md` for the `NUMBER/NUMCALF` threshold-cascade pattern used by `species_rules.m`, a good example of a data-driven (not hardcoded) validation rule.

Adding a new contributor parser

1. Copy `src/+narwc/+io/+parsers/TemplateFormat.m` to a new file in the same directory (see `NEAQFormat.m` for a filled-in example).
2. Fill in `FIELD_MAPPING` with only the native→canonical column renames you've actually confirmed against a real sample file from that contributor — don't invent a full layout you haven't verified.
3. Implement `createImportOptions()` (header row, delimiter for this contributor) and `detectFormat()`.
4. Register the class name in `ParserFactory.getAvailableParsers()`.
5. Add format metadata to `config/format_definitions.json`.
6. Run it via `narwc.ingestion.convert_contributor_batch(parser_name, input_files, output_dir)`, or the `scripts/ingestion/convert_contributor_batch.m` entry point.

Adding a config parameter

1. Add the default to `config/defaults/validation_config_default.m` under the relevant sub-struct.
2. Read it in the rule via the `config` argument.
3. Override it in a batch config (`config/batches/<name>.m`) if the default is wrong for a specific run context (e.g. `migration.m`'s permissive thresholds vs. `routine.m`'s strict ones).
4. Document it in `docs/configuration_reference.md`.

Always call `load_config()` (layered defaults → local → batch), never the older

`get_config.m` cached singleton.

Naming conventions

- Classes: PascalCase (`SurveyValidator.m`)
- Functions and processing steps: snake_case (`calculate_derived_fields.m`)
- Packages: lowercase (+validation, +processing)
- Scripts: snake_case

Testing

See `docs/testing_guide.md` for the full test-running/writing reference.
Quick version: `test_runner()` runs everything, `runtests('tests/unit/test_X.m')` runs one file, and tests requiring a live database check `tests/utils/has_live_db.m` and skip themselves when one isn't configured.